Case 03 · Research and POC: Roll Transparency Log

Following the Magshimim "Research and POC" template: basic research, five key features, and the first POC (goals, plan, design, implementation and conclusions). The POC was written in Python and actually run; the numbers here are its output. What is real and what is simulated is stated explicitly.

Basic research: sequence of events

- **Decision:** an electoral registration officer approves a change (add, delete, correct) on the officer terminal and picks a reason code from a closed list.
- **Entry:** the terminal builds an entry: sequence number, voter pseudonym (HMAC of the elector ID under a secret pepper), operation and reason, booth, date, officer pseudonym.
- **Append:** the entry becomes a leaf of the Merkle tree.
- **Signed tree head:** every few minutes the log signs its current size and root and publishes them.
- **Notice:** the voter gets an SMS with the reason code and the appeal deadline.
- **Check:** the voter (or an app on their behalf) fetches an inclusion proof and verifies it against the tree head. Monitors fetch consistency proofs between old and new heads.

Basic research: components

- **Officer Terminal:** the decision point; no roll change without an entry.
- **Log Server:** the Merkle tree, proofs and tree-head signing.
- **Pseudonym Service (in an HSM):** computes voter pseudonyms and holds the key without which a pseudonym cannot be linked to a person.
- **Voter Lookup:** portal, SMS and kiosk.
- **Monitors:** independent bodies that keep old tree heads and compare.

Basic research: security challenges

- **Silent rewrite:** the operator changes an old entry. Defence: consistency proofs and monitors holding old heads.
- **Split view:** the log shows one voter one tree and a monitor another. Defence: signed, widely distributed heads; monitors gossip.
- **Change that bypasses the log:** defence: the roll itself is derived from the log; no other change path exists.
- **Privacy:** a public log of who moved and when is information about people. Defence: irreversible pseudonyms, coarse codes, delayed publication.

Basic research: comparison with the leading alternative (the Bihar removal list)

- **Transparency:** both publish removals with reasons.
- **Integrity over time:** a list published once cannot show it was not changed later; the log catches any rewrite.
- **Notice:** with a list you must search; with the log you are told.
- **Privacy:** the Bihar list carried names; the log carries none.

Conclusion: we add proof and reverse the flow, from a voter who searches to a voter who receives.

Basic research: ethical issue

A removal log with reasons can be misused: in a small booth, an R-SHIFT line with a date tells the neighbours who left and when, and an R-DEAD line sometimes arrives before the family has told anyone.

Response: the public log shows only district and week; full detail appears only in the proof a voter receives about themselves. This is the central item of the privacy impact assessment (DPIA).

Five key features

- **Entry builder:** input: officer decision; output: canonical entry (sorted compact JSON); how: fixed fields, pseudonym instead of ID.
- **Append:** leaf `H(0x00||entry)`, node `H(0x01||L||R)` per RFC 6962.
- **Inclusion proof:** audit path per RFC 9162 §2.1.3.
- **Consistency proof:** SUBPROOF per RFC 6962 §2.1.2.
- **Signed tree head:** Ed25519 in deployment (HMAC in the POC).

The POC: research question

Can a Merkle log hold a country-sized roll, give each voter a short proof checkable on a phone, catch a rewrite of an old entry, and expose no personal data?

The POC: technologies and limits

- Pure Python, `hashlib` and `hmac` only, so it runs in the browser.
- **Important limit:** the roll, voters and changes are synthetic (fixed seed). Hashing and proofs are real. Tree-head signatures are modelled with HMAC. Server, monitors and voter run in one process.
- **Out of scope:** a real network, monitor gossip, persistent storage, SMS delivery.

The POC: flow

Officer decision → canonical entry with pseudonym → leaf → signed tree head → inclusion proof to the voter · consistency proof to monitors

The POC: implementation highlights

```
def leaf_hash(entry): return H(b"\x00" + entry)
def node_hash(l, r): return H(b"\x01" + l + r)
def voter_tag(epic): return hmac.new(SALT, epic.encode(), sha256).hexdigest()[:20]
log.append(entry)                # incremental, complete subtrees only
p = log.inclusion(m)             # audit path
verify_inclusion(leaf, m, n, p, root)
verify_consistency(m, n, log.consistency(m), old_root, new_root)
```

The POC: results

Lab	Measured	Result
Scale	1,000,000 entries	built in 18.8 s; proof 20 hashes (640 B); verify 0.03 ms
Scale	projection to 130 million	27 hashes per proof
Inclusion	Priya's entry (index 31,337 of 50,000)	found and verified, 16 hashes
Inclusion	same entry with a forged reason (R-DEAD)	proof fails
Rewrite	entry 4,242 changed after growth from 12,000 to 20,000	caught: consistency proof fails
Privacy	50,000 entries over ~41,000 booths	almost every booth has fewer than 5 changes: booth + date identify neighbours

The POC: conclusions

- ✅ **Scale:** proven with a huge margin; even at 130 million a proof is smaller than one MMS image.
- ✅ **Inclusion and rewrite:** proven; this is the heart of the idea and it works.
- ⚠️ **Privacy:** the lab's key finding. A pseudonym alone is not enough: booth and date are a quasi-identifier. Decision carried into the spec: the public log shows only district and week; exact booth and date appear only in the proof a voter receives about themselves. A separate DPIA is written.

Conclusion: ✅ survives, and moves to specification with a binding privacy requirement.